

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
import scipy.cluster.hierarchy as sch
from google.colab import files

# Set visualization style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)

# Upload your cleaned CSV file
from google.colab import files
uploaded = files.upload()

# Get the filename
filename = next(iter(uploaded))

# Load the data
import pandas as pd
df = pd.read_csv(filename)

# Display first few rows
df.head()
```

 Choose Files

cleaned_dataset.csv

- cleaned_dataset.csv(text/csv) - 468485 bytes, last modified: 4/8/2025 - 100% done

Saving cleaned_dataset.csv to cleaned_dataset.csv

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type
0	1	55	Male	Blouse	Clothing	53.0	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express
1	2	19	Male	Sweater	Clothing	64.0	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express
2	3	50	Male	Jeans	Clothing	73.0	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Standard
3	4	21	Male	Sandals	Footwear	90.0	Rhode Island	M	Maroon	Spring	3.5	Yes	PayPal	Ne...

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

```
# Basic information about the dataset
print("Dataset Info:")
print(df.info())
print("\nSummary Statistics:")
print(df.describe())
```

 Dataset Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 3897 entries, 0 to 3896

Data columns (total 19 columns):

#	Column	Non-Null Count	Dtype
0	Customer ID	3897 non-null	int64
1	Age	3897 non-null	int64
2	Gender	3897 non-null	object
3	Item Purchased	3897 non-null	object
4	Category	3897 non-null	object
5	Purchase Amount (USD)	3897 non-null	float64
6	Location	3897 non-null	object
7	Size	3897 non-null	object
8	Color	3897 non-null	object
9	Season	3897 non-null	object
10	Review Rating	3897 non-null	float64
11	Subscription Status	3897 non-null	object
12	Payment Method	3897 non-null	object
13	Shipping Type	3897 non-null	object

```

14 Discount Applied      3897 non-null object
15 Promo Code Used      3897 non-null object
16 Previous Purchases    3897 non-null float64
17 Preferred Payment Method 3897 non-null object
18 Frequency of Purchases 3897 non-null object
dtypes: float64(3), int64(2), object(14)
memory usage: 578.6+ KB
None

```

Summary Statistics:

	Customer ID	Age	Purchase Amount (USD)	Review Rating \
count	3897.000000	3897.000000	3897.000000	3897.000000
mean	1951.123428	44.080318	59.754683	3.749987
std	1125.508270	15.206247	23.734822	0.715899
min	1.000000	18.000000	0.000000	2.500000
25%	977.000000	31.000000	39.000000	3.100000
50%	1951.000000	44.000000	60.000000	3.700000
75%	2925.000000	57.000000	81.000000	4.400000
max	3900.000000	70.000000	100.000000	5.000000

	Previous Purchases
count	3897.000000
mean	25.349500
std	14.440098
min	1.000000
25%	13.000000
50%	25.000000
75%	38.000000
max	50.000000

```

# Check for missing values (though data is cleaned, good to verify)
print("Missing Values:")
print(df.isnull().sum())

```

```

Missing Values:
Customer ID      0
Age              0
Gender           0
Item Purchased   0
Category         0
Purchase Amount (USD) 0
Location         0
Size             0
Color            0
Season           0
Review Rating    0
Subscription Status 0
Payment Method   0
Shipping Type    0
Discount Applied 0
Promo Code Used  0
Previous Purchases 0
Preferred Payment Method 0
Frequency of Purchases 0
dtype: int64

```

```

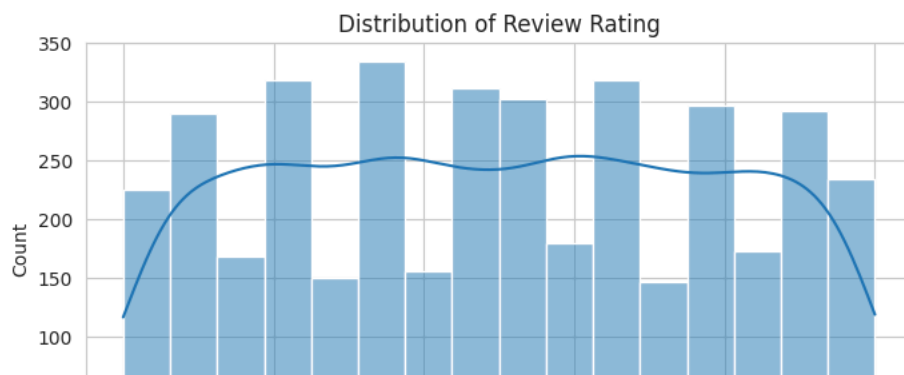
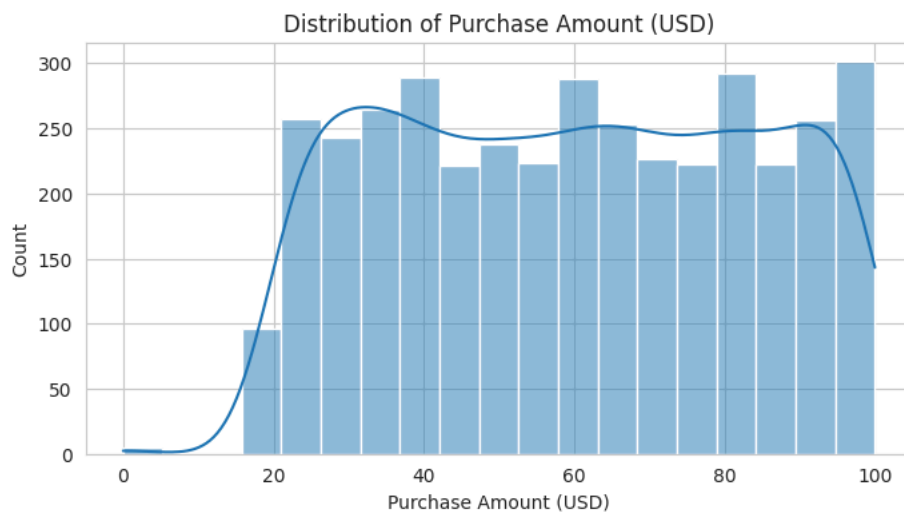
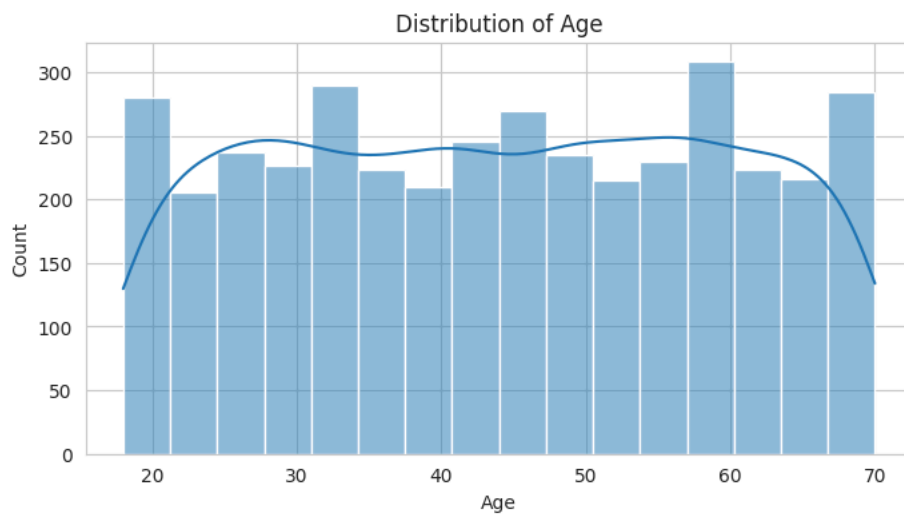
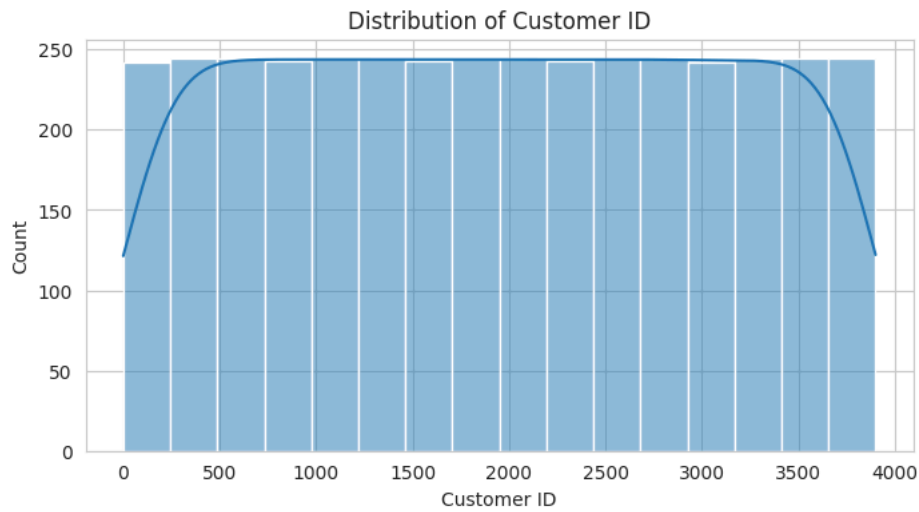
# Visualize distribution of numerical features
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns

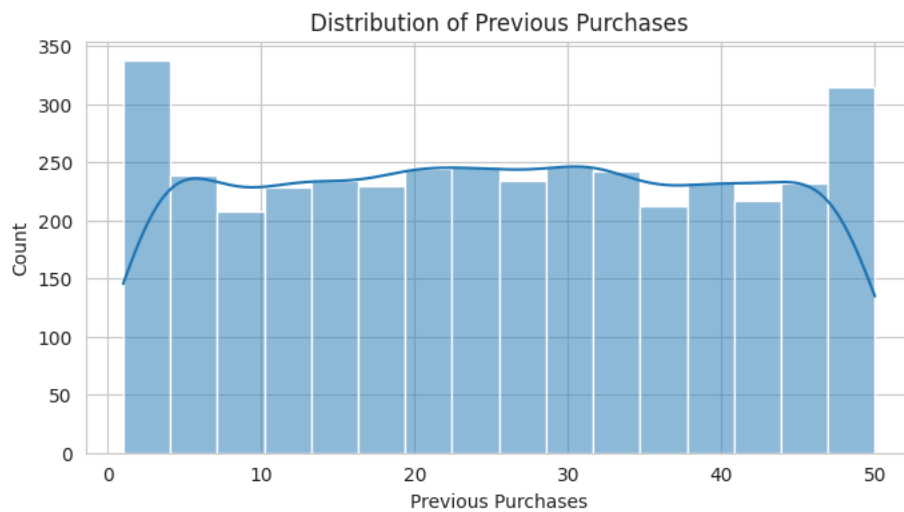
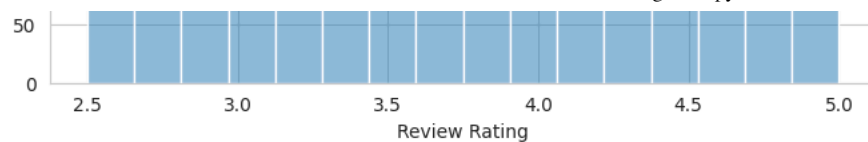
```

```

for col in numerical_cols:
    plt.figure(figsize=(8, 4))
    sns.histplot(df[col], kde=True)
    plt.title(f'Distribution of {col}')    # Shows the distribution of the graphs of each column in the dataset
    plt.show()

```



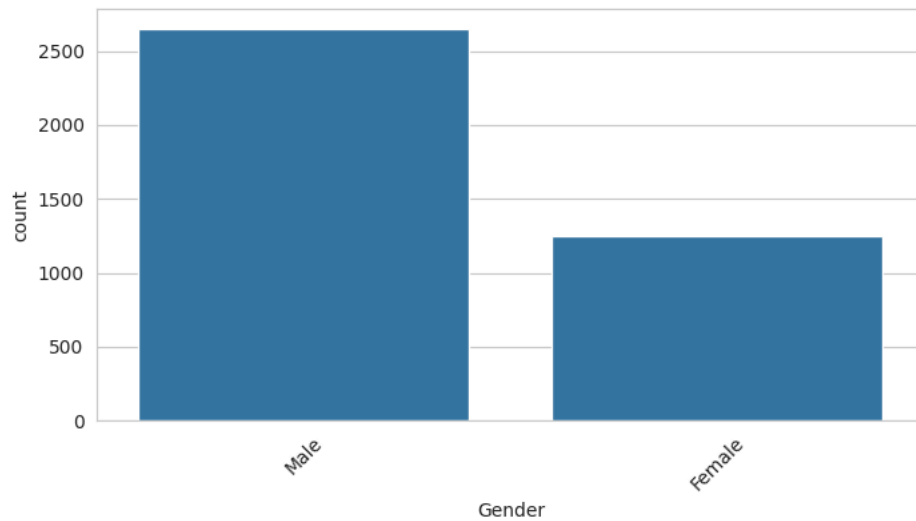


```
# Visualize categorical features
categorical_cols = df.select_dtypes(include=['object']).columns

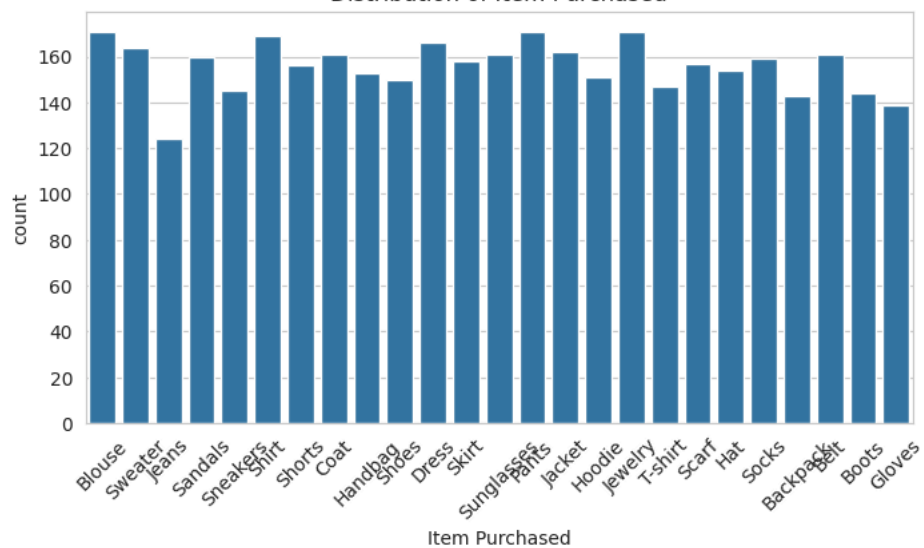
for col in categorical_cols:
    plt.figure(figsize=(8, 4))
    sns.countplot(data=df, x=col)
    plt.title(f'Distribution of {col}')      # Shows the distribution of the graphs of each column in the dataset
    plt.xticks(rotation=45)
    plt.show()
```



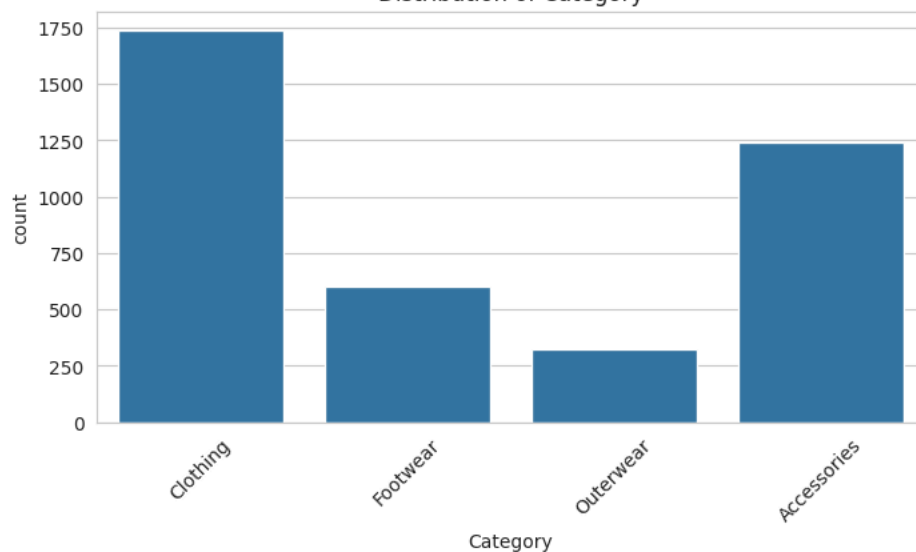
Distribution of Gender



Distribution of Item Purchased

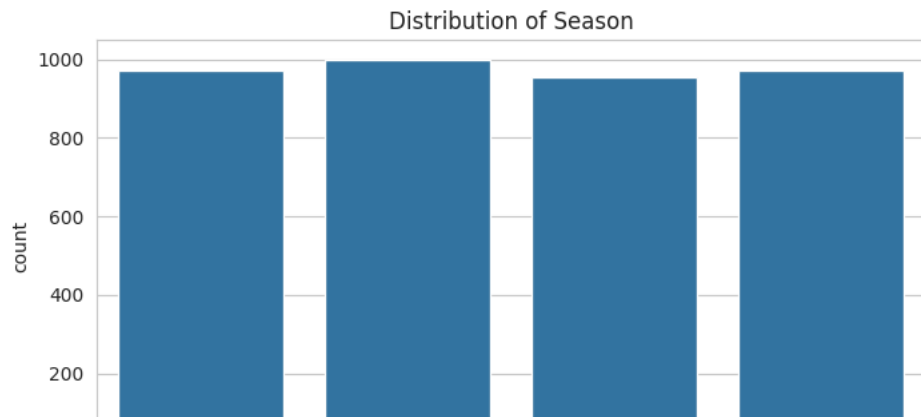
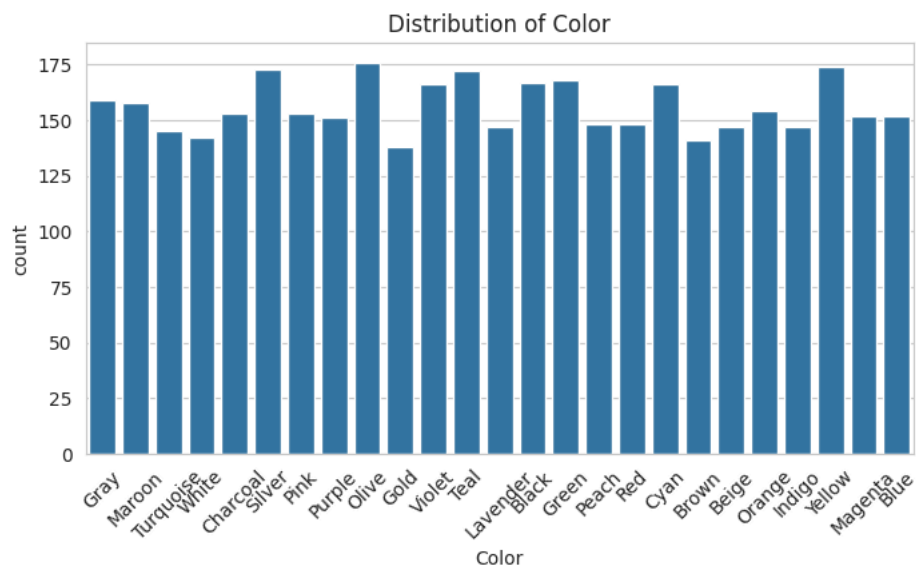
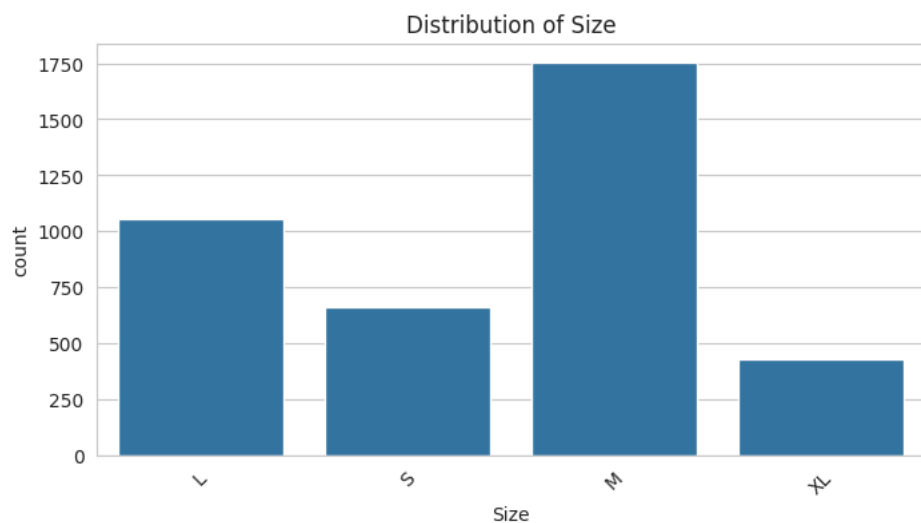
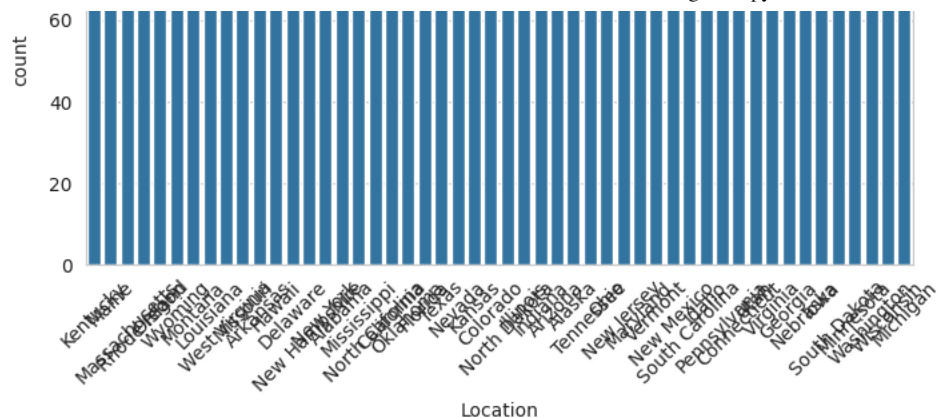


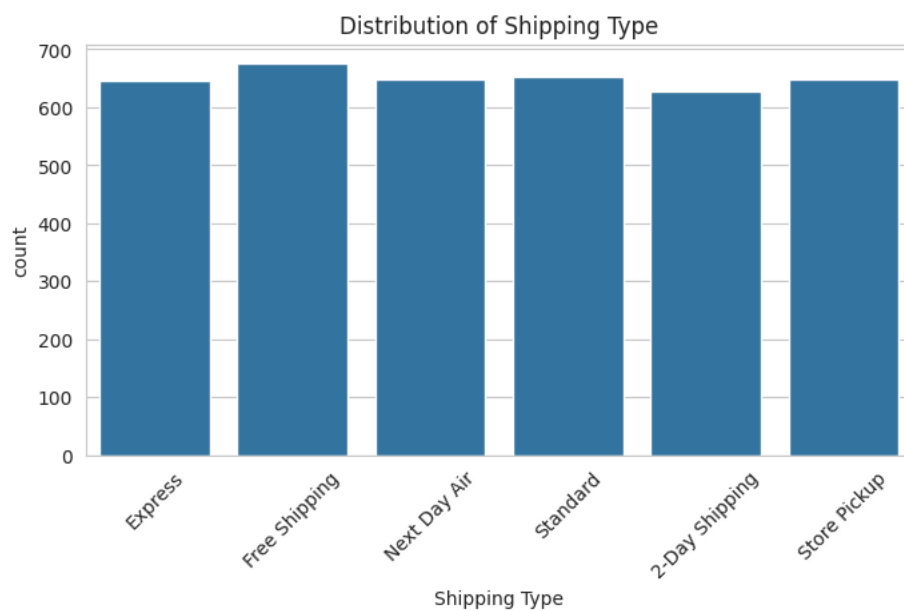
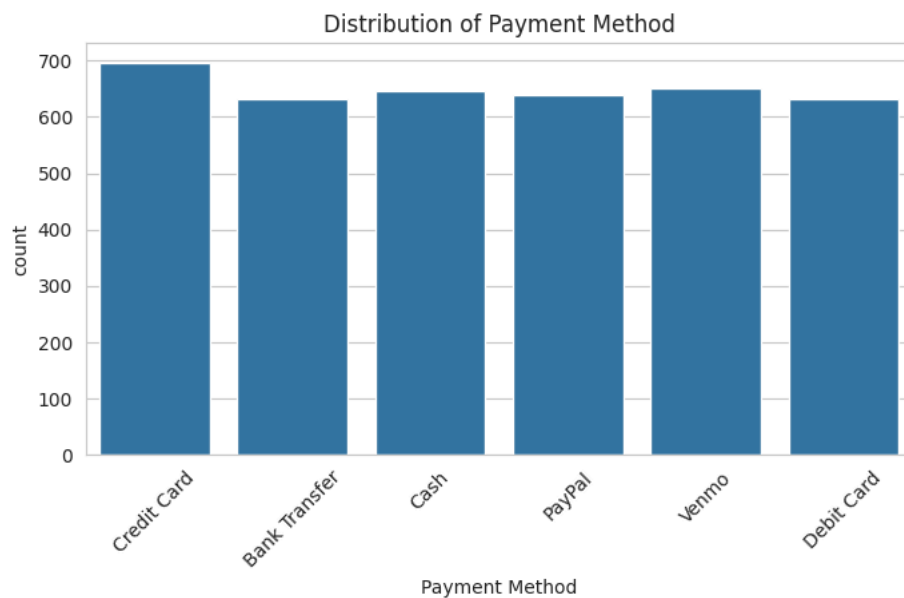
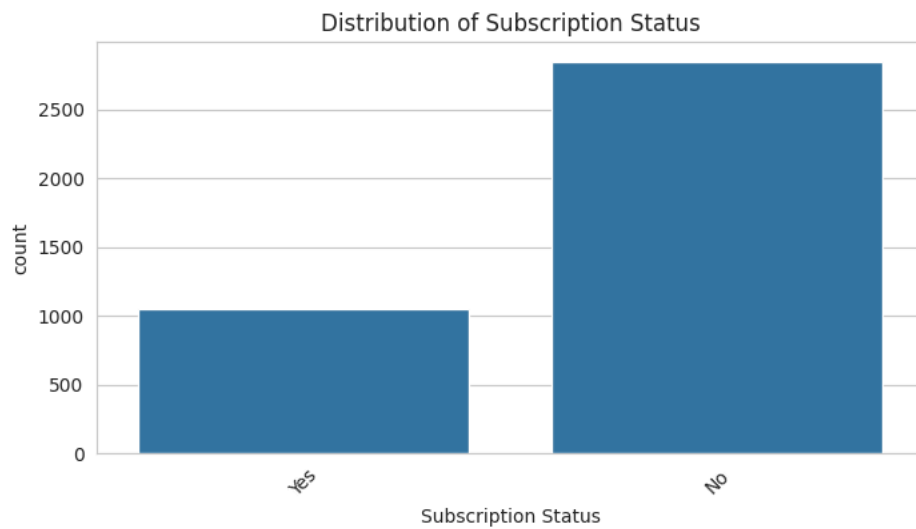
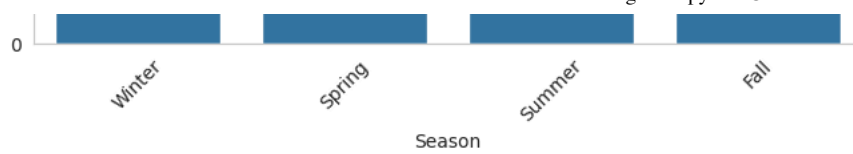
Distribution of Category



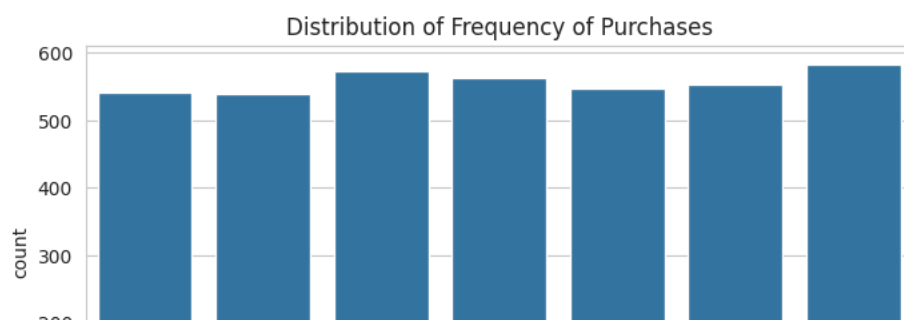
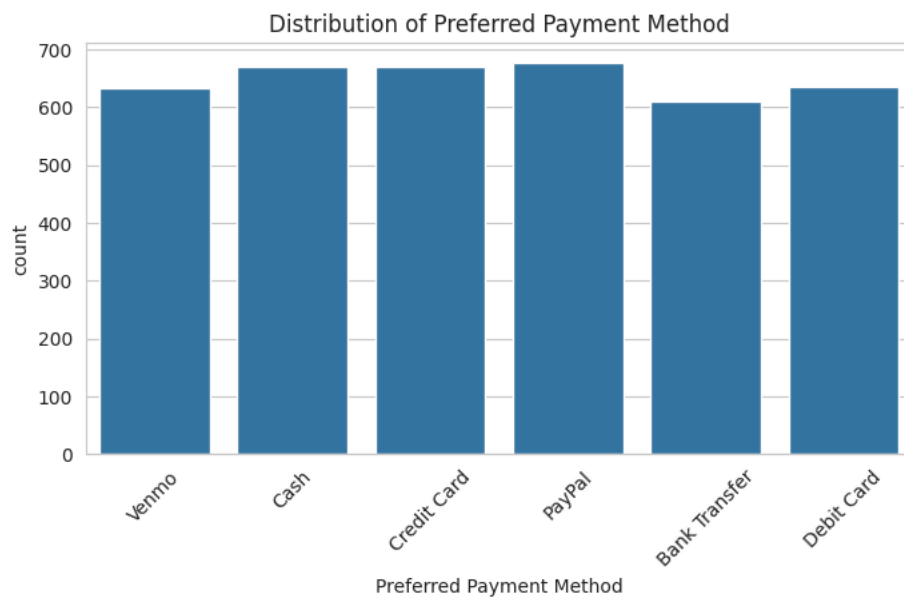
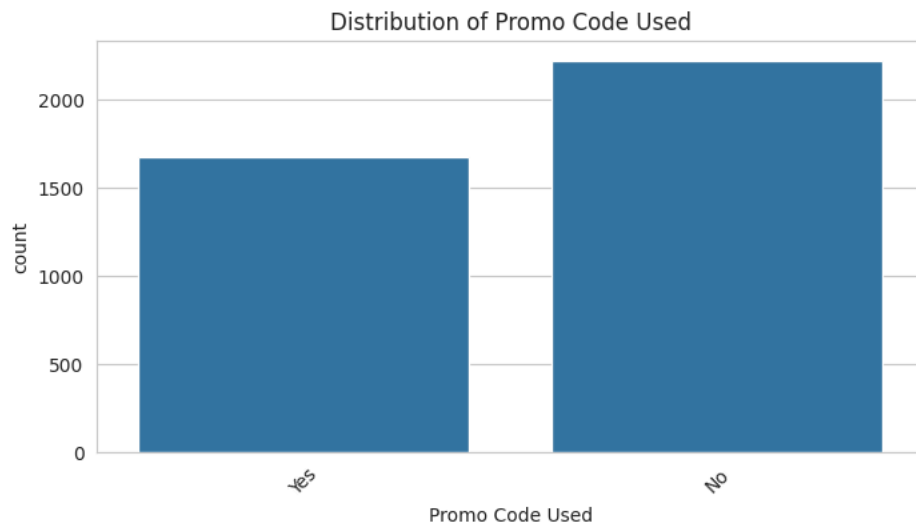
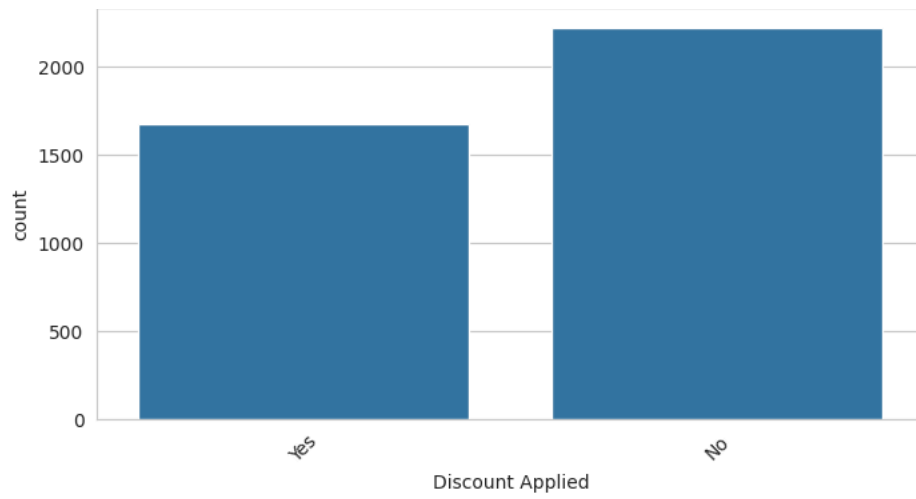
Distribution of Location

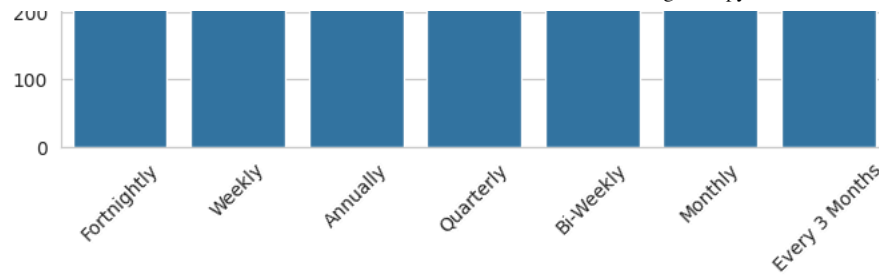




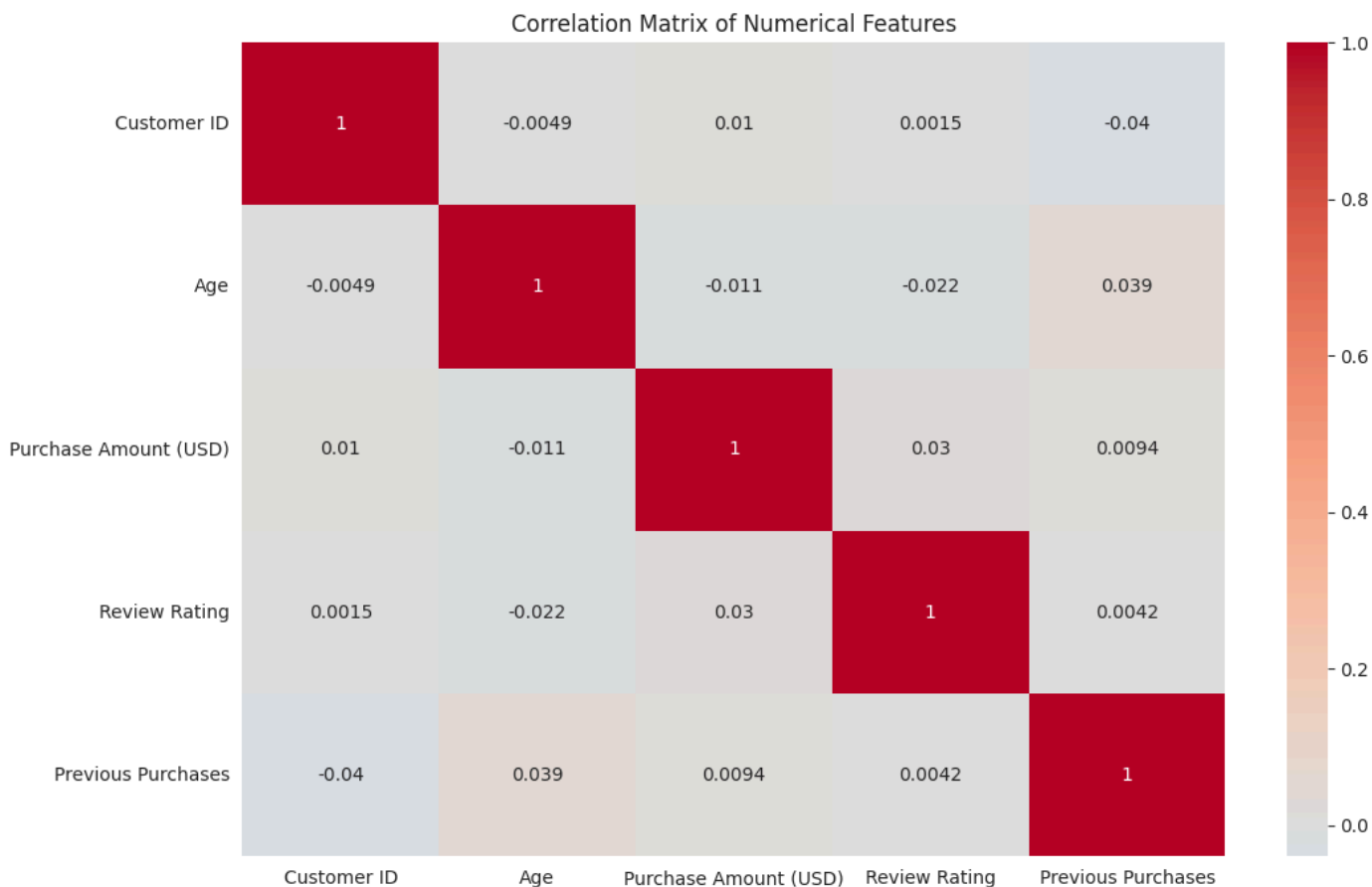


Distribution of Discount Applied





```
# Correlation matrix for numerical features
plt.figure(figsize=(12, 8))
corr_matrix = df[numerical_cols].corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```



```
# Display all column names in your DataFrame
print("Columns in your dataset:")
print(df.columns.tolist())
```

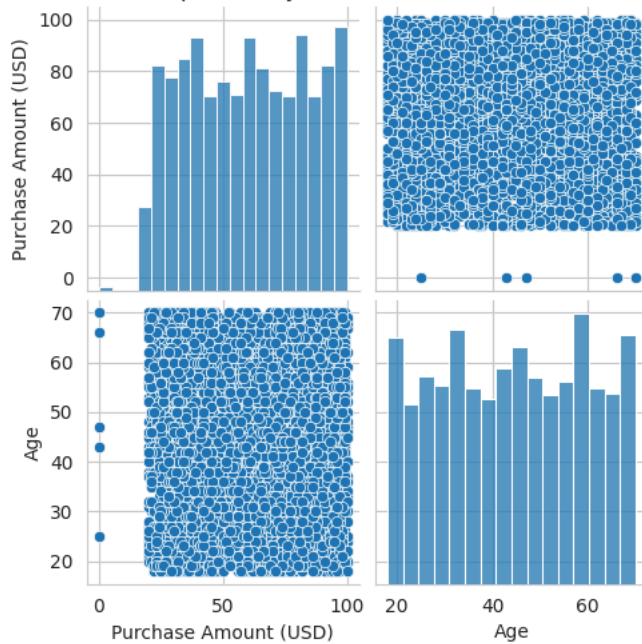


```
Columns in your dataset:
['Customer ID', 'Age', 'Gender', 'Item Purchased', 'Category', 'Purchase Amount (USD)', 'Location', 'Size', 'Color', 'Season']
```

```
# Pairplot of key numerical features (select 4-5 most important)
key_features = ['Item Purchased', 'Purchase Amount (USD)', 'Age', 'Frequency of Purchases']
sns.pairplot(df[key_features])
plt.suptitle('Pairplot of Key Numerical Features', y=1.02)
plt.show()
```



Pairplot of Key Numerical Features



```
# Select features for clustering (adjust based on your dataset)
clustering_features = ['Purchase Amount (USD)', 'Age', 'Previous Purchases']

# Extract the features
X = df[clustering_features]

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

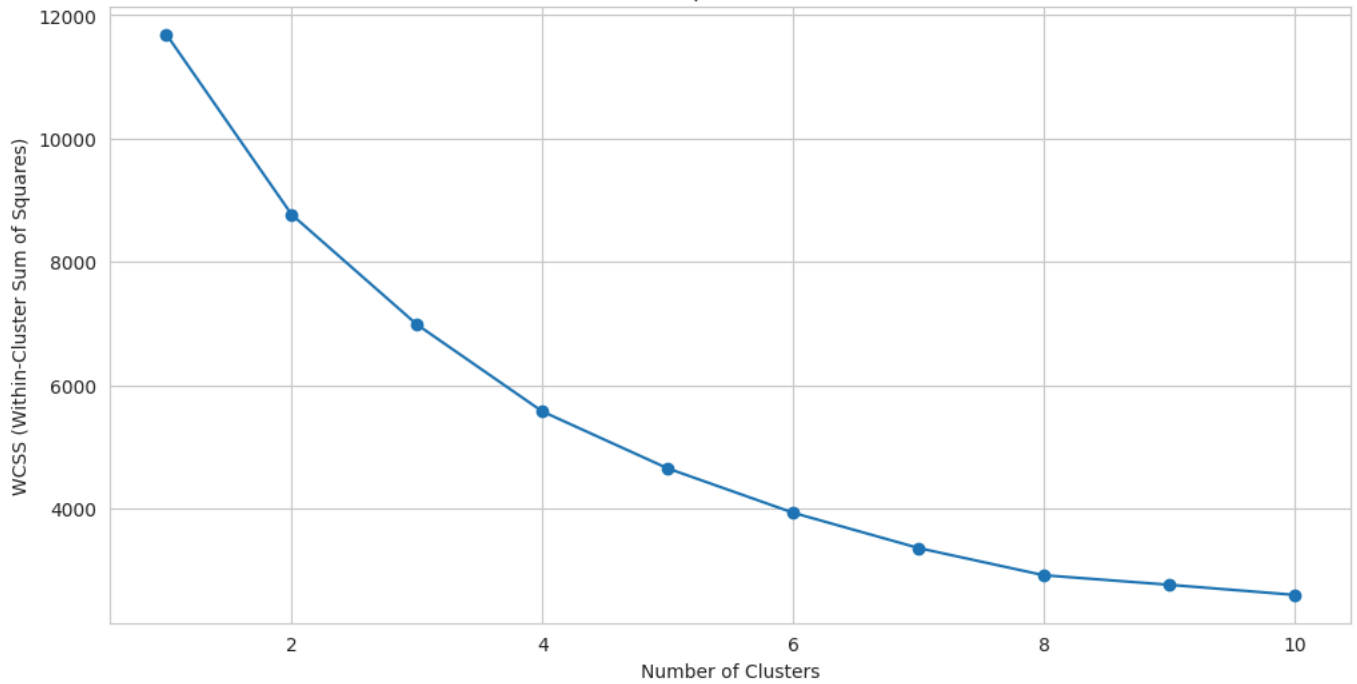
# Convert back to DataFrame for visualization
X_scaled_df = pd.DataFrame(X_scaled, columns=clustering_features)

# Elbow method to find optimal number of clusters
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

# Plot the elbow curve
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method for Optimal Number of Clusters')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS (Within-Cluster Sum of Squares)')
plt.show()
```



Elbow Method for Optimal Number of Clusters



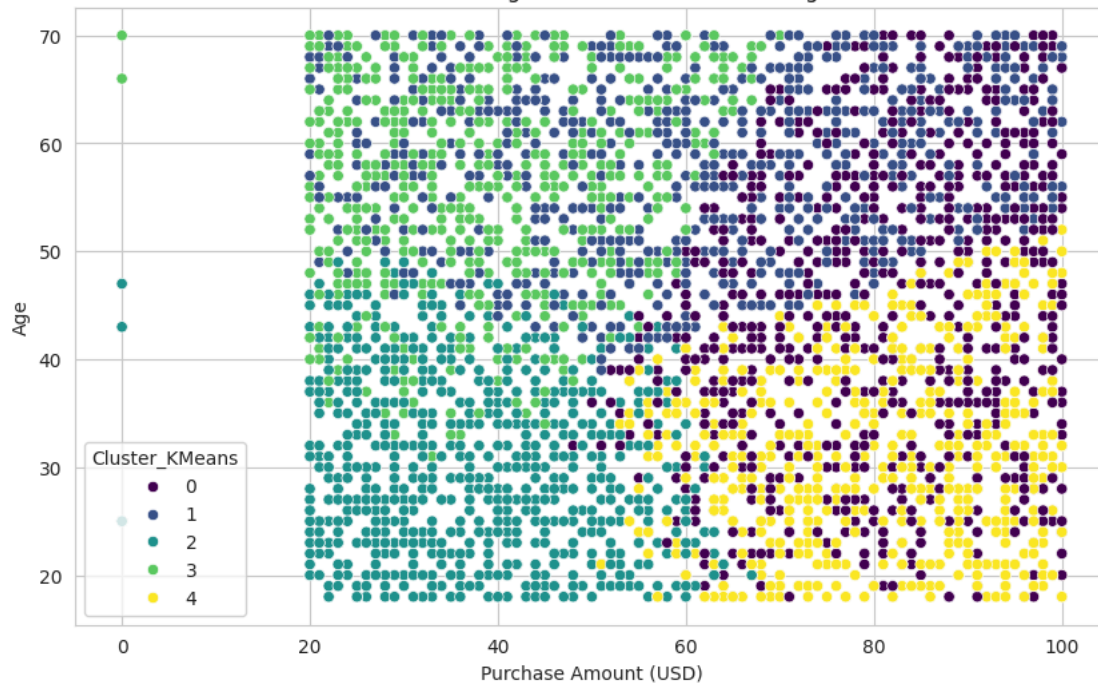
```
# Apply K-Means clustering with optimal number of clusters
optimal_k = 5 # Change based on your elbow curve
kmeans = KMeans(n_clusters=optimal_k, init='k-means++', random_state=42)
clusters_kmeans = kmeans.fit_predict(X_scaled)

# Add cluster labels to the original dataframe
df['Cluster_KMeans'] = clusters_kmeans

# Visualize the clusters
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='Purchase Amount (USD)', y='Age', hue='Cluster_KMeans', palette='viridis')
plt.title('Customer Segments - KMeans Clustering')
plt.show()
```



Customer Segments - KMeans Clustering



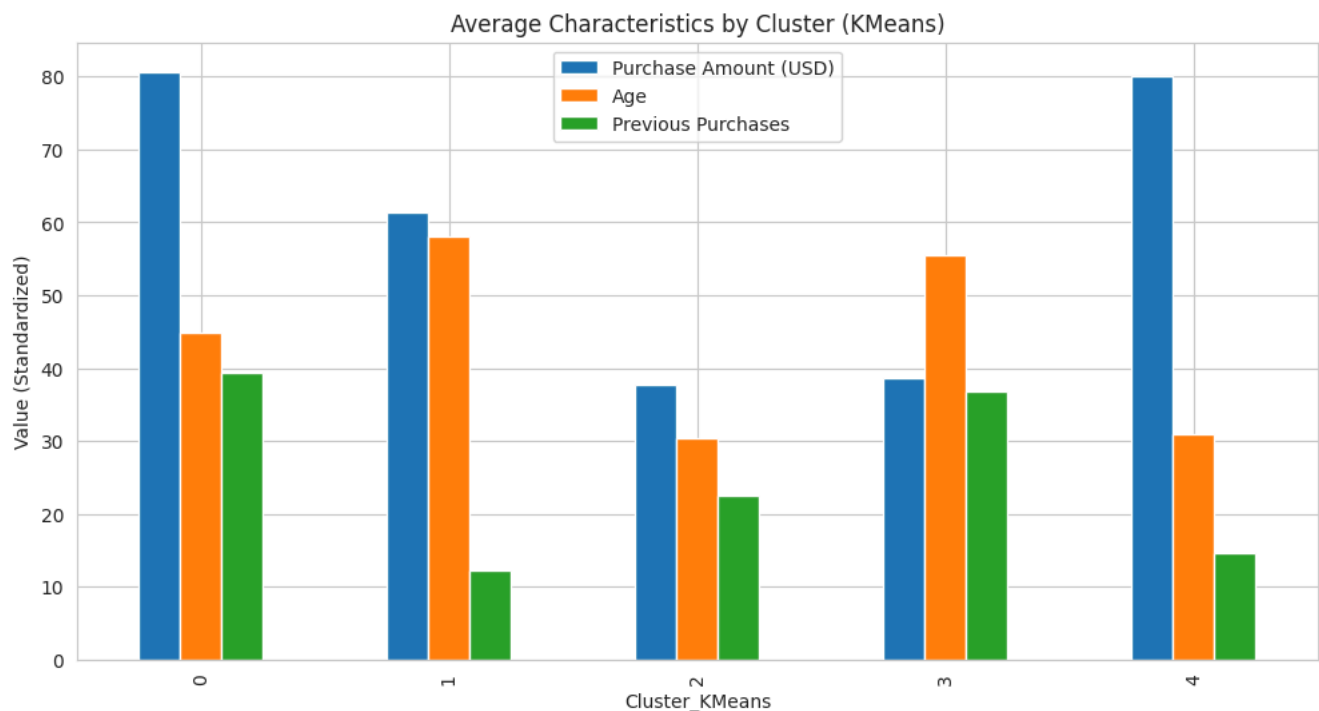
```
# Analyze cluster characteristics
cluster_profile_kmeans = df.groupby('Cluster_KMeans')[clustering_features].mean()
```

```
print("Cluster Profiles (KMeans):")
print(cluster_profile_kmeans)
```

```
# Visualize cluster profiles
cluster_profile_kmeans.plot(kind='bar', figsize=(12, 6))
plt.title('Average Characteristics by Cluster (KMeans)')
plt.ylabel('Value (Standardized)')
plt.show()
```

```
Cluster Profiles (KMeans):
```

Cluster_KMeans	Purchase Amount (USD)	Age	Previous Purchases
0	80.664352	44.841435	39.368056
1	61.385836	58.008547	12.304029
2	37.733010	30.314320	22.502427
3	38.684432	55.492286	36.760168
4	80.090103	30.995569	14.688331



```
# Dendrogram to determine number of clusters for hierarchical clustering
plt.figure(figsize=(12, 6))
dendrogram = sch.dendrogram(sch.linkage(X_scaled, method='ward'))
plt.title('Dendrogram for Hierarchical Clustering')
plt.xlabel('Customers')
plt.ylabel('Euclidean Distances')
plt.show()
```

